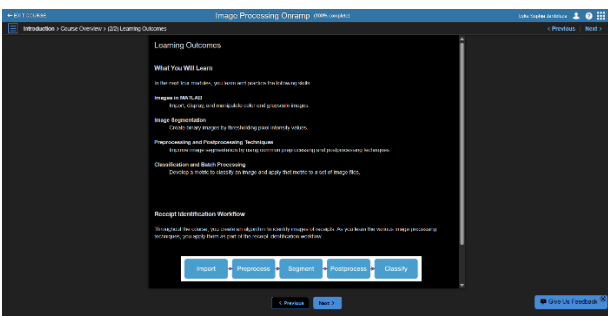


Name:	Lyka Sophia S. Jardeleza	Date:	10/31/2025
Program:	BSCpE	Section:	A461

A. Guided Exercises:

(Please include the following: 1. Photos of the codes and 2. Outputs after executing the MATLAB code.)

1. Introduction

Photo/s of MATLAB output	Photos of MATLAB Codes
	

2. Images in MATLAB

Photo/s of MATLAB output	Photos of MATLAB Codes
 Task 2  Task 3  Task 4	1. How Images are Stored in MATLAB Task 1 <pre>1 I = imread("IMG_001.jpg");</pre> Task 2 <pre>2 imshow(I)</pre> Task 3 <pre>3 I2 = imread("IMG_002.jpg");</pre> <pre>4 imshow(I2)</pre> Task 4 <pre>5 imshowpair(I,I2,"montage")</pre>



Task 1



Task 3



Task 1

```
sz = 1x2
    600    400
```

Task 2

```
sz = 1x3
    600    400     3
```

Task 2

```
Rmax = uint8
    255
```

Task 4

```
Rmin = uint8
     0
```

Task 5

2. Extract Color Planes and Intensity Values

Task 1

```
1 I = imread("IMG_003.jpg");
2 imshow(I)
```

Task 2

```
3 sz = size(I)
```

Task 3

```
4 R = I(:,:,1);
5 imshow(R)
```

Task 4

```
6 Rmax = max(R,[],"all")
```

Task 5

```
7 Rmin = min(R,[],"all")
```

3. Convert from Color to Grayscale

Do not edit. This code loads the image.

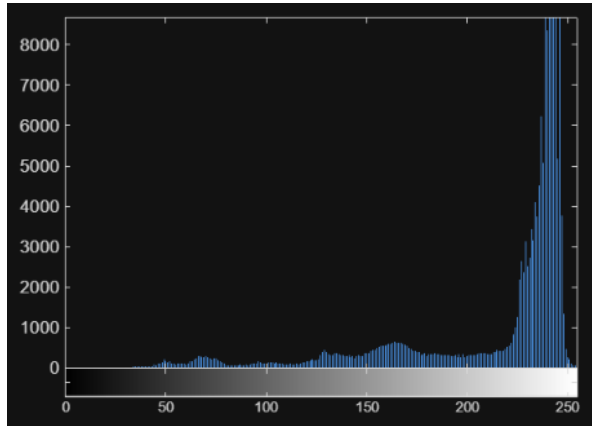
```
1 I = imread("IMG_002.jpg");
2 imshow(I)
```

Task 1

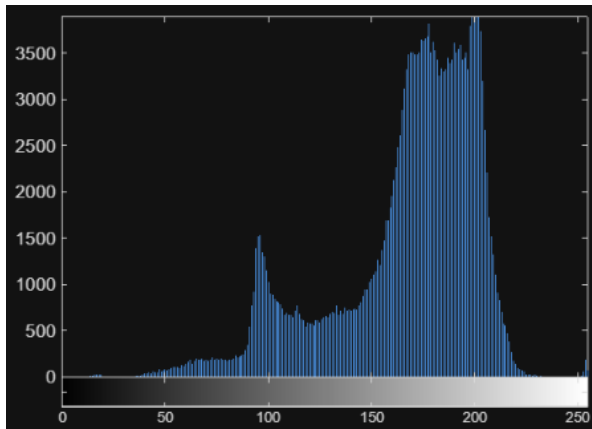
```
3 gs = im2gray(I);
4 imshow(gs)
```

Task 2

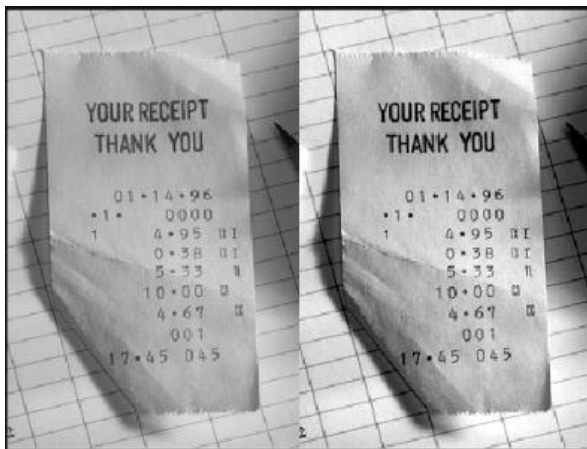
```
5 sz = size(gs)
```



Task 1



Task 2



Task 3

4. Adjust Image Contrast

Task 1

6

```
imhist(gs)
```

Task 2

7

```
imhist(gs2)
```

Task 3

8

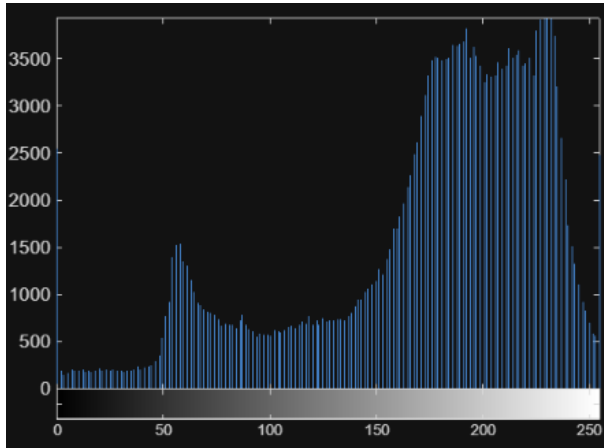
9

```
gs2Adj = imadjust(gs2);  
imshowpair(gs2,gs2Adj,"montage")
```

Task 4

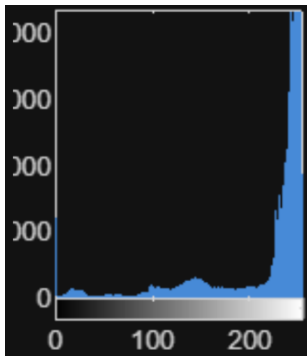
10

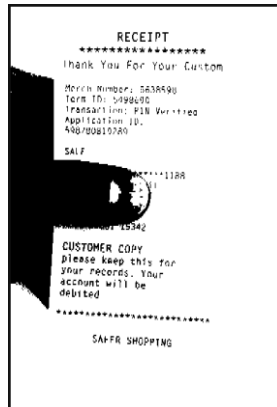
```
imhist(gs2Adj)
```



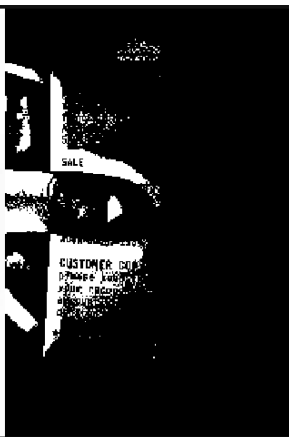
Task 4

3. Image Segmentation

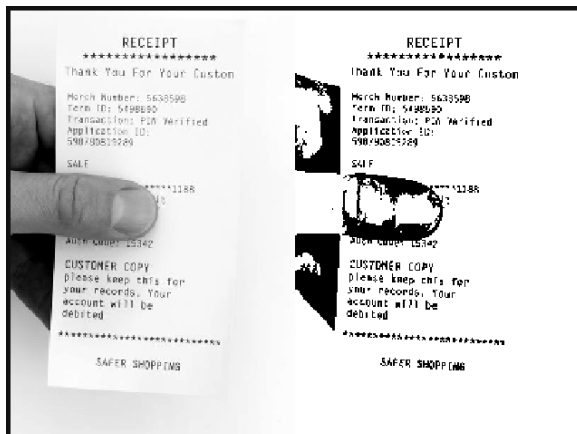
Photo/s of MATLAB output	Photos of MATLAB Codes
  Task 1  Task 2  Task 3	1. Threshold an Image Using Logical Operations <pre> 1 Do not edit. This code loads an image, converts it to grayscale, and adjusts the contrast. 2 img = imread("IMG_001.jpg"); 3 gs = im2gray(img); 4 gsAdj = imadjust(gs); 5 imshow(gsAdj) </pre> Task 1 <pre> 5 BW = gsAdj > 255/2; 6 imshow(BW) </pre> Task 2 <pre> 7 imhist(gsAdj) </pre> Task 3 <pre> 8 BW = gsAdj > 200; 9 imshow(BW) </pre>



Task 1



Task 2



Task 3

2. Automate Threshold Selection

Do not edit. This code loads an image, converts it to grayscale, and adjusts the contrast.

```
1  img = imread("IMG_001.jpg");
2  gs = im2gray(img);
3  gsAdj = imadjust(gs);
4  imshow(gsAdj)
```

Task 1

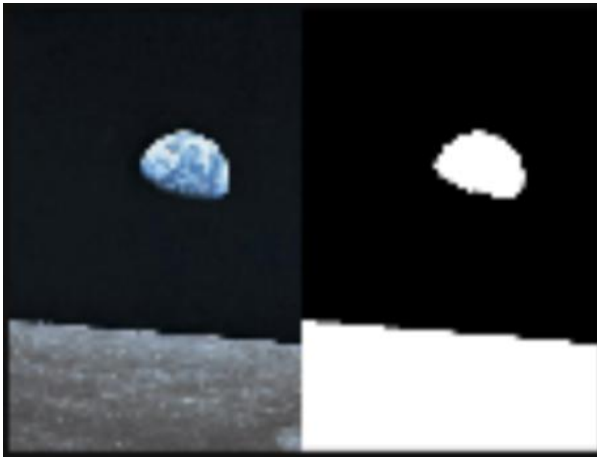
```
5  BW = imbinarize(gsAdj);
6  imshow(BW)
```

Task 2

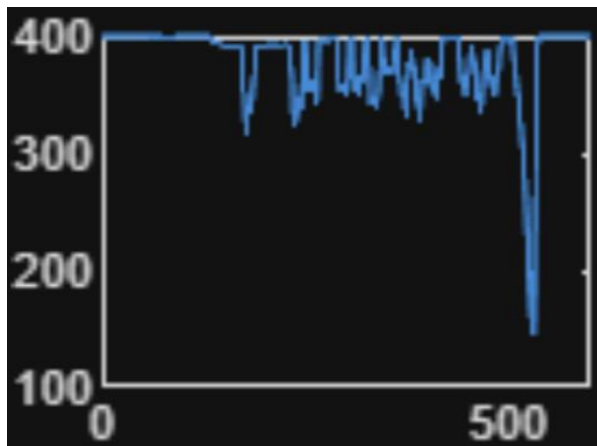
```
7  BWadapt = imbinarize(gsAdj,"adaptive");
8  imshowpair(gsAdj,BWadapt,"montage")
```

Task 3

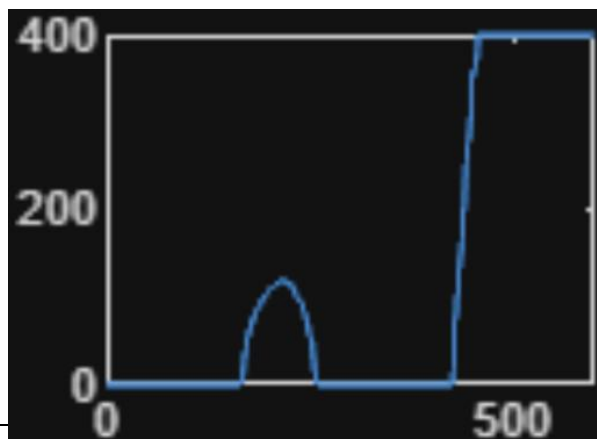
```
9  BWadapt = imbinarize(gsAdj,"adaptive","ForegroundPolarity","dark");
10 imshowpair(gsAdj,BWadapt,"montage")
```



Task 1



Task 2



3. Use a Binary Image as an Array

Do not edit. This code loads two images, processes them, and creates binary masks.

```
1 I = imread("IMG_006.jpg");
2 gs = im2gray(I);
3 gsAdj = imadjust(gs);
4 BW = imbinarize(gsAdj,"adaptive","ForegroundPolarity","dark");
5 imshowpair(I,BW,"montage")
6
7 I2 = imread("IMG_005.jpg");
8 gs2 = im2gray(I2);
9 gs2Adj = imadjust(gs2);
10 BW2 = imbinarize(gs2Adj);
11 imshowpair(I2,BW2,"montage")
```

Task 1

```
S = sum(BW,2);
```

Task 2

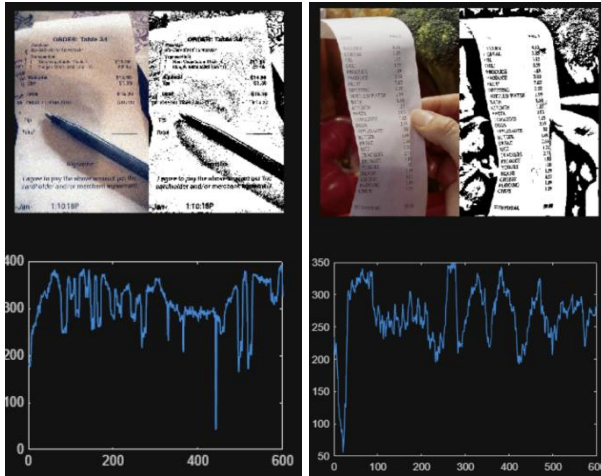
```
plot(S)
```

Task 3

```
S2 = sum(BW2,2);
plot(S2)
```

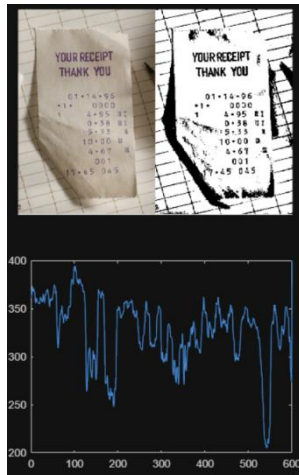


Task 3



Task 1

Task 2



Task 3

4. Compare Segmentations of Various Images

Tasks 1, 2, and 3

This code loads and processes an image. Do not edit the code. Use the drop-down list to select which image to process during each task.

```
1 imgFile = Image 7 ;  
2  
3 I = imread(imgFile);  
4 gs = im2gray(I);  
5 gsAdj = imadjust(gs);
```

Tasks 1, 2, and 3

This code loads and processes an image. Do not edit the code. Use the drop-down list to select which image to process during each task.

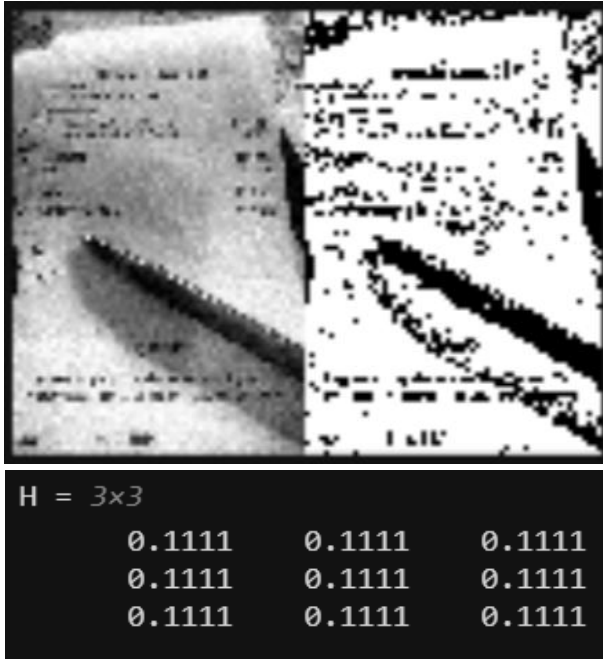
```
1 imgFile = Image 10 ;  
2  
3 I = imread(imgFile);  
4 gs = im2gray(I);  
5 gsAdj = imadjust(gs);  
6 BW = imbinarize(gsAdj, 'adaptive', 'ForegroundPolarity', 'dark');  
7 imshowpair(I, BW, 'montage')  
8 S = sum(BW, 2);  
9 plot(S)  
10
```

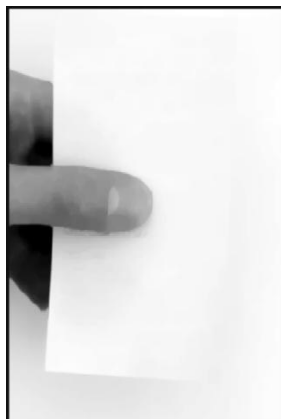
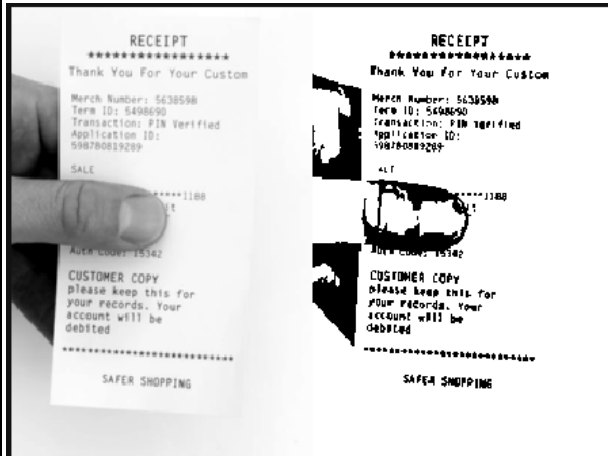
Tasks 1, 2, and 3

This code loads and processes an image. Do not edit the code. Use the drop-down list to select which image to process during each task.

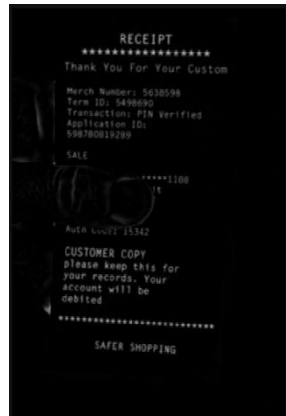
```
1 imgFile = Image 2 ;  
2  
3 I = imread(imgFile);  
4 gs = im2gray(I);  
5 gsAdj = imadjust(gs);  
6 BW = imbinarize(gsAdj, 'adaptive', 'ForegroundPolarity', 'dark');  
7 imshowpair(I, BW, 'montage')  
8 S = sum(BW, 2);  
9 plot(S)  
10
```

4. Preprocessing and Postprocessing Techniques

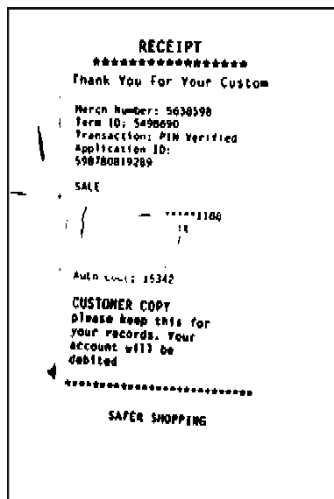
Photo/s of MATLAB output	Photos of MATLAB Codes
 Task 1	1. Remove Noise from an Image Do not edit. This code loads and processes an image. <pre> 1 I = imread("IMG_007.jpg"); 2 gs = im2gray(I); 3 gs = imadjust(gs); 4 BW = imbinarize(gs,"adaptive","ForegroundPolarity","dark"); 5 imshowpair(gs,BW,"montage") </pre> Task 1 <pre> 6 H = fspecial("average",3) </pre> Task 2 <pre> 7 gssmooth = imfilter(gs,H); </pre> Task 3 <pre> 8 BWsmooth = imbinarize(gssmooth,"adaptive","ForegroundPolarity","dark"); 9 imshow(BWsmooth) </pre> Task 4 <pre> 10 gssmooth = imfilter(gs,H,"replicate"); </pre> Task 5 <pre> 11 BWsmooth = imbinarize(gssmooth,"adaptive","ForegroundPolarity","dark"); 12 imshow(BWsmooth) </pre>
 Task 3	
 Task 5	



Task 1



Task 3



Task 4

2. Remove the Background from a Receipt Image

Do not edit. This code loads and processes IMG_001.jpg and stores the result in gs.

```
1 I = imread("IMG_001.jpg");  
2 gs = im2gray(I);  
3 gs = imadjust(gs);  
4 H = fspecial("average",3);  
5 gs = imfilter(gs,H,"replicate");  
6  
7 BW = imbinarize(gs,"adaptive","ForegroundPolarity","dark");  
8 imshowpair(gs,BW,"montage")
```

Task 1

```
9 SE = strel("disk",8);
```

Task 2

```
10 Ibg = imclose(gs,SE);  
11 imshow(Ibg)
```

Task 3

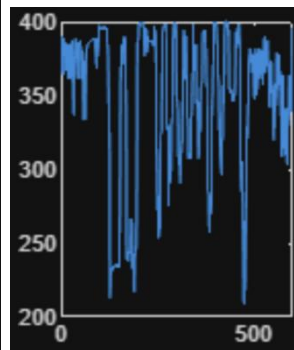
```
12 gsSub = Ibg - gs;  
13 imshow(gsSub)
```

Task 4

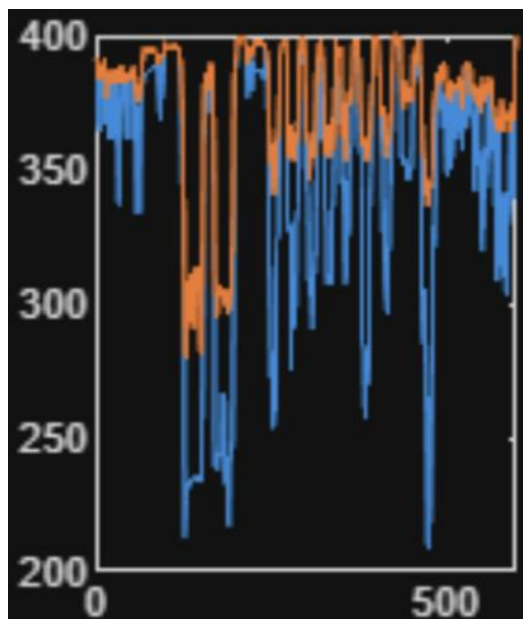
```
14 BWsub = ~imbinarize(gsSub);  
15 imshow(BWsub)
```



Task 2



Task 3



Task 4

3. Augment the Text Pattern in a Receipt Image

```

1  I = imread("IMG_002.jpg");
2  gs = im2gray(I);
3  gs = imadjust(gs);
4  H = fspecial("average",3);
5  gs = imfilter(gs,H,"replicate");
6  SEdisk = strel("disk",8);
7  Ibg = imclose(gs,SEdisk);
8  gsSub = Ibg - gs;
9  BW = ~imbinarize(gsSub);
10 imshowpair(I,BW,"montage")

```

Task 1

```

11 SE = strel("rectangle",[3 25]);

```

Task 2

```

12 BWstripes = imopen(BW,SE);
13 imshow(BWstripes)

```

Task 3

```

14 S = sum(BWstripes,2);
15 plot(S)

```

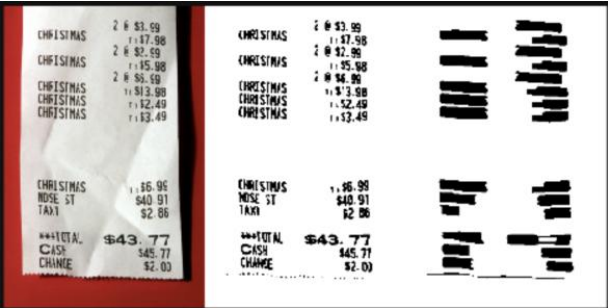
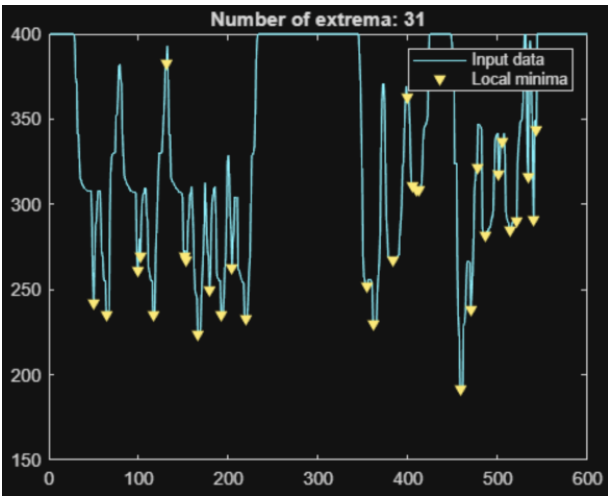
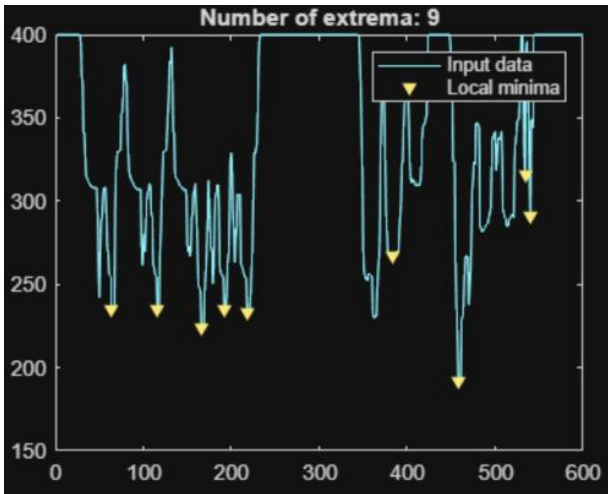
Task 4

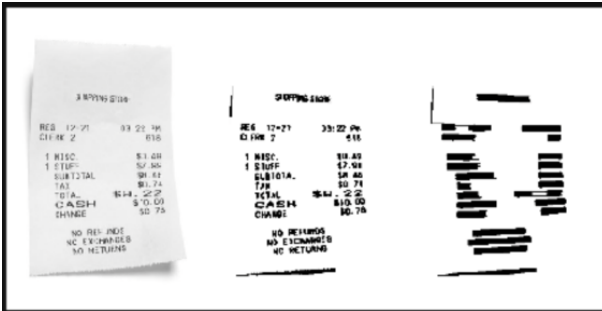
```

16 Sbw = sum(BW,2);
17 hold on
18 plot(Sbw)
19 hold off

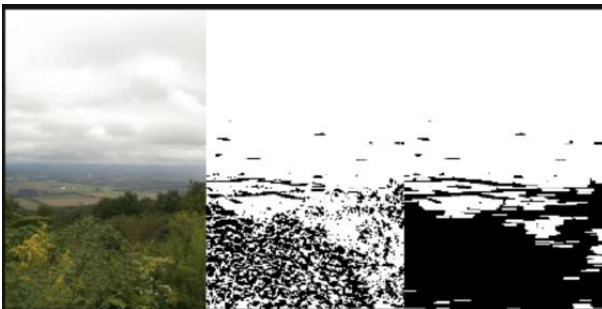
```

5. Classification and Batch Processing

Photo/s of MATLAB output	Photos of MATLAB Codes
 Task 1  Task 2  Task 3	1. Find Local Minima in a Signal Tasks 1, 4, and 5 This code loads, processes, and displays an image. Do not edit the code. Use the drop-down list to select which image to process during each task. <pre> 1 I = imread('Image3'); % Load an image 2 [S,BW,Bwstrips] = processImage(I); % Process the image to obtain a signal 3 montage([I,BW,Bwstrips]) % Show the image </pre> Task 1 Tasks 2 and 3 Find Local Extrema minIndices = Local minima in S Select data Input data: S X-axis: default Define local extrema Extrema type: Minima Flat selection: Center Max num extrema: 600 Min prominence: 0 Min separation: 0 Prominence window: Centered Prominence: 100 Display results ☒ Input data ☒ Local minima Show code Task 2 Tasks 2 and 3 Find Local Extrema minIndices = Local minima in S Select data Input data: S X-axis: default Define local extrema Extrema type: Minima Flat selection: Center Max num extrema: 600 Min prominence: 70 Min separation: 0 Prominence window: Centered Prominence: 25 Display results ☒ Input data ☒ Local minima Show code Task 3 Tasks 1, 4, and 5 This code loads, processes, and displays an image. Do not edit the code. Use the drop-down list to select which image to process during each task. <pre> 1 I = imread('Image6'); % Load an image 2 [S,BW,Bwstrips] = processImage(I); % Process the image to obtain a signal 3 montage([I,BW,Bwstrips]) % Show the image </pre> Task 4 Tasks 1, 4, and 5 This code loads, processes, and displays an image. Do not edit the code. Use the drop-down list to select which image to process during each task. <pre> 1 I = imread('Image8'); % Load an image 2 [S,BW,Bwstrips] = processImage(I); % Process the image to obtain a signal 3 montage([I,BW,Bwstrips]) % Show the image </pre> Task 5



Task 4



Task 5

`nMin = 16`

Task 1

`isReceipt = logical
1`

Task 2

```
ds =  
ImageDatastore with properties:  
    Files: {  
        '/users/mwa0800030741016/.training/.work/testimages/IMG_001.jpg';  
        '/users/mwa0800030741016/.training/.work/testimages/IMG_002.jpg';  
        '/users/mwa0800030741016/.training/.work/testimages/IMG_003.jpg';  
        ... and 15 more  
    }  
    Folders: {  
        '/users/mwa0800030741016/.training/.work/testimages'  
    }  
    AlternateFileSystemRoots: {}  
    ReadSize: 1  
    Labels: {}  
    SupportedOutputFormats: ["png" "jpg" "jpeg" "tif" "tiff"]  
    DefaultOutputFormat: "png"  
    ReadFcn: @readDatastoreImage
```

Task 1

2. Apply the Classification Metric to an Image

19	Task 1 <code>nMin = nnz(minIndices)</code>
20	Task 2 <code>isReceipt = nMin >= 9</code>

3. Create an Image Datastore

1	Task 1 <code>ds = imageDatastore("testimages")</code>
2	Task 2 <code>dataFileNames = ds.Files</code>
3	Task 3 <code>nFiles = numel(dataFileNames)</code>

```
dataFileNames = 18x1 cell
'/users/mwa0000030741016/.training/.work/testimages/IMG_001.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_002.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_003.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_004.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_005.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_006.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_007.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_008.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_009.jpg'
'/users/mwa0000030741016/.training/.work/testimages/IMG_010.jpg'
:
```

Task 2

nFiles = 18

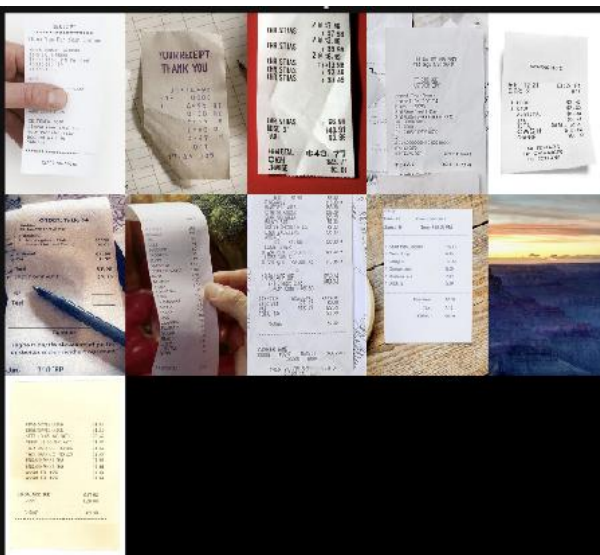
Task 3



Task 1

**isReceipt = logical
1**

Task 2



4. Read and Classify an Image

	Do not edit. This code creates the datastore ds.
1	<code>ds = imageDatastore("testimages");</code>
2	Task 1
3	<code>I = readimage(ds,1); imshow(I)</code>
4	Task 2
	<code>isReceipt = classifyImage(I)</code>

5. Extract Images that Contain Receipts

1	<code>ds = imageDatastore("testimages");</code>
2	<code>nFiles = numel(ds.Files);</code>
3	<code>isReceipt = false(1,nFiles);</code>
4	Task 1
5	<code>for k = 1:nFiles</code>
6	<code> I = readimage(ds,k);</code>
7	<code> isReceipt(k) = classifyImage(I);</code>
	<code>end</code>
8	Task 2
	<code>receiptFiles = ds.Files(isReceipt);</code>
9	Task 3
10	<code>montage(receiptFiles) title("Receipts")</code>

6. Conclusion

Photo/s of MATLAB output

Summary: Image Processing Onramp

Image Processing Onramp Workflow

Throughout this lab, you learned various image processing techniques and applied them as part of a complete classification system.

Import Image Preprocess Segment Postprocess Classify

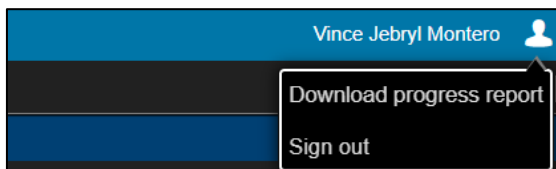
Function Reference

Function	Description	Example
<code>imread</code>	Load an image into the workspace.	<code>I = imread('myImage.png');</code>
<code>imshow</code>	Display an image in the figure.	<code>imshow(I);</code>
<code>subplot</code>	Display two images as an overlay in the figure.	<code>subplot(2,1,1), imshow(I);</code>
<code>hold on</code>	Display multiple images next to each other.	<code>hold on; imshow(I2, 'hold on');</code>
<code>rgb2gray</code>	Convert an RGB image to grayscale.	<code>gr = rgb2gray(I);</code>
<code>size</code>	Display the dimensions of a grayscale image.	<code>size(gr);</code>
<code>find</code>	Return the coordinates of an image.	<code>[m, n] = find(I > 0.5);</code>

Photos of MATLAB Codes

B. MATLAB Onramp Progress Report

Please attach the completed MATLAB Onramp progress report. You can access the progress report by clicking on the name icon at the top right corner, then click the Download progress report.




MathWorks | Training Services

Progress Report

Name: Lyka Sophia Jardeleza
Course: Image Processing Onramp
Progress: 100% complete (as of 31 October 2025)

Chapters

1. Introduction **100%**
2. Images in MATLAB **100%**
3. Image Segmentation **100%**
4. Preprocessing and Postprocessing Techniques **100%**
5. Classification and Batch Processing **100%**
6. Conclusion **100%**

Release: v1 | Language: English



C. Findings ,Observations, and Comments:

Guiding Questions :

1. *What did you learn in the guided exercises? Did you learn something new?*
2. *What are your findings, observations, and comments in the guided exercises?*
3. *What are your findings, observations, and comments in the problem exercises?*

In the guided exercises, I learned how MATLAB can be used for image processing and analysis. I gained a better understanding of how images are stored as matrices and how different operations can be applied to enhance or modify them. I also learned new techniques such as extracting color planes, converting images to grayscale, and performing segmentation. These lessons helped me connect image processing concepts to real applications in digital signal processing.

During the guided exercises, I observed how each MATLAB function works step by step in processing images. I noticed that changing certain parameters, such as contrast or threshold levels, can greatly affect the quality and clarity of the output. The exercises also showed how important it is to use the right sequence of image operations, like preprocessing before segmentation, to get accurate results. Overall, the guided tasks were straightforward but helped reinforce the fundamentals of image handling in MATLAB.

In the problem exercises, I found that combining multiple techniques was necessary to achieve better image results. I also noticed that some problem exercises required more logical thinking and experimentation with code to produce the expected outputs. These tasks helped me become more confident in troubleshooting and applying MATLAB functions to solve practical image processing problems.